



ZEALYNX

Web3 Security and AI Agent & MCP Audits

Yieldcoin v2

Smart Contract Audit

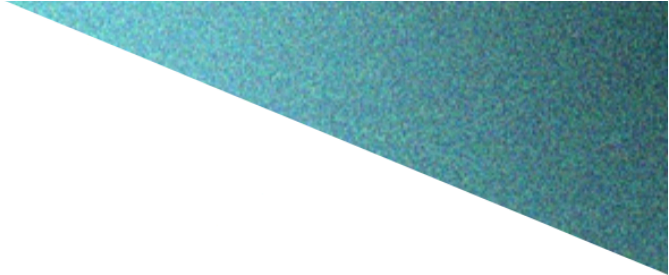
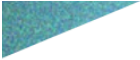
15 September 2026

Zealynx

contact@zealynx.io

Carlos Vendrell (Bloqarl)

Lead Security Auditor



Contents

1. About Zealynx

2. Disclaimer

3. Overview

3.1 Project Summary

3.2 About Yieldcoin v2

3.3 Audit Scope

4. Audit Methodology

5. Severity Classification

6. Executive Summary

6.1 The Key Findings

6.2 Architectural Security Observations

6.3 Security Strengths Observed

7. Audit Findings

7.1 Critical Severity Findings

7.2 High Severity Findings

7.3 Medium Severity Findings

7.4 Low Severity Findings

1. About Zealynx

Zealynx, founded in January 2024 by Carlos (Bloqarl), specializes in smart contract audits, and development. Our services include comprehensive smart contract audits, application security audits, such as pentesting, and AI Audits. We are trusted by clients such as Badger DAO, Ample protocol, Lido, Inverter, Matchain, and Golden Grid.

Our team believes in transparency and actively contributes to the community by creating educational content. This includes challenges for Foundry and Rust, security tips for Solidity developers, and fuzzing materials like Foundry and Echidna/Medusa. We also publish articles on topics such as preparing for an audit and battle testing smart contracts on our website [Zealynx.io](https://zealynx.io) and our blogs.

Zealynx has achieved public recognition, including winning 1st prize in the Uniswap Hook Hackathon and a Top 5 position in the Beanstalk Audit public contest. Our ongoing commitment is to provide value through our expertise and innovative security solutions for smart contracts.

2. Disclaimer

A security review of a smart contract can't guarantee there are no vulnerabilities. It's a limited effort in terms of time, resources, and expertise to find as many vulnerabilities as possible. We can not assure 100% security post-review or guarantee the discovery of any issues with your smart contracts.

3. Overview

3.1 Project Summary

Yieldcoin v2 is a crosschain stablecoin yield optimizer. A single ParentVault holds the accounting while ChildVaults on other chains hold the position, and capital moves between them over Chainlink CCIP. Deposits and withdrawals are batched into epochs and settled at one price per epoch, driven by Chainlink CRE automation.

3.2 About Yieldcoin v2

The protocol keeps one active strategy at a time, in an Aave v3, Aave v4 or Compound v3 market that may sit on the parent chain or on a child chain. Users submit deposit and withdraw intents into an open epoch; closeEpoch settles the whole epoch against a TVL observation supplied by the CRE workflow, fixes the price, and nets the two sides so only the difference crosses a chain boundary. Users later claim the shares or assets that settlement allocated them.

Three lifecycles are mutually exclusive by construction — epoch settlement, rebalancing, and recovery — and each cross-chain operation carries an explicit nonce checked against vault state. Where a ChildVault operation fails after value has already moved, it stores typed recovery that anyone may retry, so no privileged party is needed to unstick funds.

3.3 Audit Scope

Solidity sources under evm/src/, at frozen commit a08ecdfdb5aa603366ac9a342791cc532052ee52. The Chainlink CRE Go workflow, off-chain services, deployment scripts, tests and third-party dependencies were out of scope, except where they affect in-scope contracts.

```
evm/src/vaults/ParentVault.sol
evm/src/vaults/ChildVault.sol
evm/src/vaults/BaseVault.sol
evm/src/libraries/vaults/ParentVaultEpochLib.sol
evm/src/libraries/vaults/ParentVaultUserEpochLib.sol
evm/src/libraries/vaults/ParentVaultCcipLib.sol
evm/src/libraries/vaults/ParentVaultRebalanceLib.sol
evm/src/libraries/vaults/ParentVaultFeesLib.sol
evm/src/libraries/vaults/ParentVaultMathLib.sol
evm/src/libraries/vaults/ParentVaultConfigLib.sol
evm/src/libraries/vaults/BaseVaultCcipLib.sol
evm/src/libraries/vaults/BaseVaultConfigLib.sol
evm/src/libraries/vaults/BaseVaultStrategyLib.sol
evm/src/modules/WorkflowRouter.sol
evm/src/modules/ProtocolAdapter.sol
evm/src/modules/AdapterRegistry.sol
```

evm/src/modules/adapters/AaveV3Adapter.sol
evm/src/modules/adapters/AaveV4Adapter.sol
evm/src/modules/adapters/CompoundV3Adapter.sol
evm/src/token/YieldcoinShare.sol
evm/src/libraries/Types.sol
evm/src/libraries/Roles.sol

4. Audit Methodology

Approach

During our security assessments, we uphold a rigorous approach to maintain high-quality standards. Our methodology encompasses thorough **Fuzz** and **Invariant Testing**, and **meticulous manual security review**.

Throughout the smart contract audit process, we prioritize the following aspects to uphold excellence:

- 1. Code Quality:** We diligently evaluate the overall quality of the code, aiming to identify any potential vulnerabilities or weaknesses.
- 2. Best Practices:** Our assessments emphasize adherence to established best practices, ensuring that the smart contract follows industry-accepted guidelines and standards.
- 3. Documentation and Comments:** We meticulously review code documentation and comments to ensure they accurately reflect the underlying logic and expected behavior of the contract.

5. Severity Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

6. Executive Summary

This was a **two-week review** by a single auditor against a frozen commit, covering roughly 3,000 nSLOC of in-scope Solidity. Findings were filed as issues during the engagement so remediation could begin before this report, and a **mitigation review** was carried out afterwards against the client's fixes.

Where a defect could be demonstrated mechanically a test was written and then controlled: the cited line was corrected and the test re-run, so each proof depends on the defect rather than on the harness. Several proofs fork mainnet at a pinned block and run against deployed upstream bytecode rather than mocks. **Nine of the eleven findings are backed by such a test.**

A review of fixed duration cannot be exhaustive, and the absence of a finding in an area is not assurance about that area. Effort concentrated on the seams — adapter integration and the parent/child boundary — which produced seven of the eleven findings.

The engagement surfaced **11 findings: 1 Critical, 1 High, 6 Medium and 3 Low**. At the mitigation review, eight were fixed, two mitigated with documented residuals, and one accepted as a known issue.

6.1 The Key Findings

- **A signed report could be replayed at a receiver it was never intended for.** The forwarder took the receiver as a call argument rather than a signed field, so the same signatures were valid at any router. A report authorising a withdrawal on one child could be redirected to drain a different, larger position.
- **One base unit of USDC could permanently freeze the protocol.** Epoch netting can produce a one-unit remote deposit, which the destination lending market rejects because it converts to zero shares. The rejection was stored as replayable recovery, so every retry met the same permanent boundary while recovery exclusivity blocked all other work.
- **Shares were minted against value that never arrived.** A successful cross-chain send established that a message was accepted, not that the amount was delivered. Where a token pool deducted a transfer fee, existing holders absorbed the shortfall silently, with no revert, no recovery and nothing to alert on.
- **Recovery treated every failure as temporary.** Three findings share one design decision: typed recovery replays the original operation with its original parameters, which is correct for a fee spike or a temporarily illiquid market and unrecoverable when the obstacle can never clear.

6.2 Architectural Security Observations

The dominant weakness was one design decision expressed in several places rather than a scatter of unrelated defects: **recovery treated every caught failure as transient**. It stores the original operation with its original parameters and replays exactly that. Three findings are instances — a transfer above a lane's capacity, a supply that rounds to zero shares, and a deposit blocked by a cap the vault itself consumed. In each the stored recovery was not a path back to progress but a guarantee that the vault could not leave the state it was in.

What made this architectural rather than three bugs is that the distinction already existed in the code and was simply not applied to failure classification. `ChildVault._ccipSend` deliberately hoists its validation outside the `try`, commented *"so configuration errors revert instead of being stored as recovery."* The line was drawn one step too early: a permanently unsatisfiable operation is a configuration error in every sense that matters, and it was being captured.

A second pattern ran through the cross-chain accounting: **a call that returned successfully was treated as an outcome that completed**. A successful send means a message was accepted, not that value arrived. A third concerned **guards asymmetric between mirrored operations** — a deposit floor with no withdrawal counterpart, and withdraw settlement reconciling to delivered value while deposit settlement did not.

6.3 Security Strengths Observed

The recovery architecture is the right shape for the problem, and the findings concern how failures are classified within it rather than whether it should exist. Making `executeRecovery` permissionless is the decision worth calling out: no privileged party is required to unstick funds, which materially reduces dependence on operator availability. The split between the parent reverting atomically and the child storing recovery is reasoned in writing, on the grounds that parent-side failures leave no cross-chain state another party is waiting on — a distinction many protocols never articulate.

Nonce discipline is unusually careful, and it defeated an attack path during this engagement rather than merely surviving review: a candidate finding was withdrawn because the completion function derives the nonce itself and rejects any argument that does not match. The surrounding guards held under the same pressure. Inbound messages validate the source chain before touching value, senders are bound to registered counterpart vaults, and adapters are bound to their vault and reject a mismatched asset.

The project's security documentation is the strongest encountered on a first external audit, and it was load-bearing for the work. Several of the most valuable findings exist precisely because the team wrote down what the system assumes — including accepted residual risks, stated plainly rather than elided — which made those assumptions testable.

Summary of Findings:

Vulnerability	Severity	Status
[C-01] Receiver-unbound Child reports can drain a later active strategy	Critical	Fixed
[H-01] A one-unit Aave v4 deposit permanently pins a remote epoch	High	Fixed
[M-01] Failed close-report retries let late depositors capture incumbent yield	Medium	Fixed
[M-02] Adapter donation recovery lets an unseeded bootstrapper steal depositor principal	Medium	Mitigated
[M-03] Outbound CCIP recovery is reported as backing while committed to a settled epoch	Medium	Fixed
[M-04] Over-capacity CCIP transfer is stored as replayable recovery, permanently locking every holder's principal	Medium	Fixed
[M-05] Base-unit withdrawals exhaust Child LINK and pin global settlement	Medium	Mitigated
[M-06] CCIP token-pool fee updates silently undercollateralize remote-deposit shares	Medium	Fixed
[L-01] ParentVault beneficiary erases withdrawals and strands all redeemed assets	Low	Fixed
[L-02] Inactive Aave target supply can self-seal a full rebalance recovery	Low	Acknowledged
[L-03] Report-driven completion of a Parent-local rebalance target can lock backing in transit	Low	Fixed

7. Audit Findings

7.1 Critical Severity Findings

[C-01] Receiver-unbound Child reports can drain a later active strategy

Affected files

- `evm/src/modules/WorkflowRouter.sol#L106–L145`
- `evm/src/vaults/ChildVault.sol#L220–L253`
- `evm/src/vaults/ChildVault.sol#L621–L645`
- `evm/src/libraries/vaults/ParentVaultCcipLib.sol#L55–L82`
- `evm/src/libraries/vaults/ParentVaultEpochLib.sol#L356–L366`

Description

`WorkflowRouter.onReport` authenticates the workflow ID, name, owner, and selector, but it does not authenticate the chain or router for which the report was produced. This matters because `KeystoneForwarder.report(receiver, rawReport, reportContext, signatures)` is public and its signature digest covers only `rawReport` and `reportContext`; `receiver` is caller-selected after signature verification. Keystone also keys replay state by `(receiver, workflowExecutionId, reportId)`, so success at one receiver does not consume the same report at another receiver.

Every Child router is configured to accept the same workflow identity and `executeEpochWithdraw` selector. After an honest `executeEpochWithdraw(e, amount)` report has settled an epoch through Child A, its report bytes, context, and signatures are public. If a later ordinary rebalance activates Child B, B's rebalance high-water advances but its independent epoch high-water can remain below `e`. An unprivileged caller can submit the unchanged signed report to B's Forwarder/router. B accepts `e`, withdraws amount from its current adapter, and sends a second `EPOCH_NET_WITHDRAW` message for the old epoch.

The source CCIP send succeeds, so Child B records no Yieldcoin recovery. The Parent has already made epoch `e` claimable and permanently rejects the immutable second payload in `_finalizeEpoch`. CCIP can retry the destination execution, but every retry reaches the same terminal Parent state.

The staging deployment independently confirms the complete primitive. Base Sepolia transaction `0x35025414a798fa1fff584d1614ee9ee4e8bb6aedc08910e4975cdec73743edf6` contains a successful signed `executeEpochWithdraw(3,999986)` report. At Optimism Sepolia block 47,635,395, read-only execution of those exact bytes through its configured Forwarder accepts all four signatures, calls the other Child, reduces its live Compound adapter TVL from 3,500,014 to 2,500,028, and constructs a second nonce-3 CCIP message. At Arbitrum Sepolia block 299,497,773, Parent epoch nonce is 4 and epoch 3 is already CLAIMABLE.

Where this sits relative to the existing protections

None of the checks on this path are missing or misconfigured. `WorkflowRouter` performs exactly the authentication it is built to perform — Keystone forwarder role, registered workflow ID, name and owner,

and the selector allowlist for the current generation — and every one of them passes here, because the report genuinely originates from the correct workflow and carries an allowlisted selector. The gap is on an axis the router does not consider at all: which chain, and which router, the report was produced for.

The Child nonce high-water is the same. Accepting any nonce strictly greater than the last handled one is deliberate and correct: a Child does not participate in every global epoch, so its nonce sequence is legitimately sparse and skipped values must be tolerated. That design defends the ordering it was built for — commands arriving late, or with gaps. It does not defend a Child whose high-water has never advanced against a historical command it was never intended to receive at all.

The protocol already enforces origin binding on its other cross-chain ingress path. An inbound CCIP message is checked against the immutable router callback, the source chain selector, and the cross-chain vault registered for that selector, so a message is bound to the place it came from. Reports arriving through the CRE path receive no equivalent binding, because the Keystone signature quorum is treated as sufficient authorization on its own. That asymmetry between the two ingress paths is what this finding exploits.

Vulnerable scenario:

1. CRE honestly signs and delivers `executeEpochWithdraw(e, amount)` to active Child A; the transaction calldata becomes public.
2. A normal rebalance later moves the remaining strategy to Child B. B has material TVL, but its epoch high-water is still below `e`.
3. The attacker calls B's public Keystone Forwarder with B's router as `receiver` and the unchanged report, context, and signatures from step 1.
4. Child B withdraws `amount` from its live adapter and CCIP accepts an old-epoch message.
5. Parent rejects that message forever because epoch `e` is already claimable.

Impact

An unprivileged attacker can remove a historical withdrawal amount from a later active Child strategy and strand it behind permanently failing CCIP destination execution. The amount can be a material portion of vault principal and scales with an ordinary historical withdrawal; the attacker supplies no assets and permanently spends only transaction gas.

The displaced funds are not merely delayed. The Parent's rejection of the duplicated nonce is terminal, so no retry, recovery, pause cycle, route restoration, or later epoch can route them back; returning them requires a bespoke upgrade rather than any existing operational path.

`ChildVault.executeRebalance` carries the same exposure today. Its guards have the identical shape — role, pause, reentrancy, no-recovery, and a rebalance nonce strictly greater than that Child's rebalance high-water — with no binding to the chain or router the command was produced for. Because a rebalance withdraws the active strategy's entire position rather than one historical epoch amount, a redirected rebalance command displaces the whole balance of the targeted Child. A Child that has sat idle through several rebalances holds a correspondingly low rebalance high-water, which is the ordinary sparse-nonce state the design intends to tolerate.

Recommendations

There are two ways to close this, with different cost profiles. They are not exclusive.

Option A — give each Child chain its own workflow identity. The redirect survives only because every Child router accepts one shared workflow ID, name and owner, so a report authenticated for one chain authenticates on all of them. Registering a distinct identity per Child chain makes the router's existing metadata check reject a foreign report with no contract change; this is confirmed by re-running the proof of concept with a distinct workflow ID registered on the second Child's router, where the redirected delivery no longer executes.

This is not free. A single scheduled workflow currently drives epoch settlement and rebalances across every chain, so distinct per-chain identities mean splitting it into per-chain workflows, with the operational and cost implications that carries. It is offered as a decision, not as an obviously cheaper fix.

Option B — bind the command to its destination in the signed payload. Append (targetChainId, targetRouter) to the signed report, validate both in WorkflowRouter, and strip the suffix before calling the vault. The CRE report producer must append the intended values before report generation so they fall under the existing signature digest. This keeps a single workflow identity and places the guarantee in the contract rather than in deployment discipline.

```
diff --git a/evm/src/interfaces/modules/IWorkflowRouter.sol
b/evm/src/interfaces/modules/IWorkflowRouter.sol
@@
+     error WorkflowRouter__ReportTooShort(uint256 reportLength);
+     error WorkflowRouter__InvalidReportDomain(uint256 targetChainId,
address targetRouter);
diff --git a/evm/src/modules/WorkflowRouter.sol
b/evm/src/modules/WorkflowRouter.sol
@@
import {IWorkflowRouter} from
"../interfaces/modules/IWorkflowRouter.sol";
+import {IChildVault} from "../interfaces/vaults/IChildVault.sol";
@@
+     if (report.length < 4) revert
WorkflowRouter__ReportTooShort(report.length);
+     bytes4 selector = bytes4(report[:4]);
+     bytes calldata vaultCall = report;
+
+     if (selector == IChildVault.executeEpochWithdraw.selector) {
+         // selector + two uint256 arguments + chain ID + router
address
+         if (report.length != 132) revert
WorkflowRouter__ReportTooShort(report.length);
+         uint256 targetChainId = uint256(bytes32(report[68:100]));
+         address targetRouter =
```

```

address(uint160(uint256(bytes32(report[100:132]))));
+         if (targetChainId != block.chainid || targetRouter !=
address(this)) {
+             revert WorkflowRouter__InvalidReportDomain(targetChainId,
targetRouter);
+         }
+         vaultCall = report[:68];
+     }
@@
-         (bool success, bytes memory returnData) = i_vault.call(report);
+         (bool success, bytes memory returnData) =
i_vault.call(vaultCall);

```

The diff above covers executeEpochWithdraw only. executeRebalance needs the same treatment in the same change, not as later work: it is reachable today by the identical redirect and displaces the targeted Child's entire position rather than a single epoch amount. Any future CRE-driven Child command should carry the domain binding from the outset.

If Keystone itself can be changed, including `block.chainid` and the receiver address directly in its signed digest is the strongest generic protection, since it removes the caller's freedom to choose the destination for an already-signed report rather than compensating for it downstream.

Mitigation

Fixed in 14a57d6: reports now carry a signed envelope naming the target chain and router, so the receiver is part of the signature digest and a redirected report is rejected on `WorkflowRouter__InvalidReportDomain`.

7.2 High Severity Findings

[H-01] A one-unit Aave v4 deposit permanently pins a remote epoch

Affected files

- `evm/src/modules/adapters/AaveV4Adapter.sol#L51–L60`
- `evm/src/libraries/vaults/ParentVaultEpochLib.sol#L192–L275`
- `evm/src/vaults/ChildVault.sol#L133–L145, #L390–L414`
- `evm/src/vaults/BaseVault.sol#L278–L300, #L359–L364`
- `evm/lib/aave-v4/src/spoke/Spoke.sol#L225–L240`
- `evm/lib/aave-v4/src/hub/Hub.sol#L200–L220`
- `evm/lib/aave-v4/src/hub/libraries/SharesMath.sol#L13–L27`

Description

At frozen revision `a08ecdadb5aa603366ac9a342791cc532052ee52`, `AaveV4Adapter.deposit` assumes every positive asset amount can be supplied. It approves the configured Spoke, calls `supply(reserveId, amount, address(this))`, and only checks the position delta after that call returns.

The real Aave v4 Hub has a stricter semantic boundary. `Spoke.supply` transfers the assets to the Hub and calls `Hub.add`; the Hub converts assets to added shares by rounding down and rejects zero shares:

```
uint120 shares = asset.toAddedSharesDown(amount).toUint120();
require(shares > 0, InvalidShares());
```

With Aave v4's virtual balances, the conversion is:

```
floor(amount * (addedShares + 1_000_000) / (addedAssets + 1_000_000))
```

The configured Ethereum USDC reserve already has `addedAssets > addedShares`, so a one-unit USDC

`supply` produces zero shares and deployed Aave bytecode reverts `InvalidShares()`. The adapter's fixed 100-unit discrepancy tolerance cannot handle this because the upstream call reverts before the post-deposit check.

Yieldcoin can send this amount even though each user deposit must be at least one USDC.

`ParentVaultEpochLib` aggregates deposits and floor-valued withdrawals and calculates

`netFlow = totalDepositAmount - totalWithdraw`. A one-USDC deposit netted against a withdrawal

worth `0.999999` USDC therefore creates a remote net deposit of exactly one base unit. Parent marks the epoch `EXECUTING` and bridges that unit.

When the Child receives it, `_handleCCIPDeposit` catches the adapter revert and stores `EPOCH_DEPOSIT(epochNonce, 1)`. `_recoverFailedEpochDeposit` can only retry the same one-unit

amount. Aave v4 maintains a non-decreasing added-asset/share exchange rate, so once the configured reserve is above 1:1, normal accrual and elapsed time cannot make one asset unit mint a share.

Meanwhile `_requireNoRecovery` rejects later Child operations, and Parent refuses to close another epoch or initiate a rebalance while the prior epoch is executing.

Vulnerable scenario:

1. The active strategy is the configured Ethereum Aave v4 USDC reserve.
2. A user submits the minimum one-USDC deposit while a holder submits shares whose floor-valued withdrawal claim is `0.999999` USDC.
3. The epoch operator closes the epoch with a truthful TVL report. Parent computes and sends one USDC base unit and records the epoch as `EXECUTING`.
4. Production CCIP accepts the amount and delivers it to Child.
5. The real Aave v4 Hub rounds the supply to zero shares and reverts `InvalidShares()`.
6. Child stores exact one-unit deposit recovery. Every recovery attempt reaches the same permanent share-conversion boundary, while the recovery and executing-epoch guards prevent all normal progress.

No Aave governance action, reserve cap exhaustion, donation, market illiquidity, or protocol insolvency is required.

KI-014 accepts a superficially similar shape: recovery retries the full original amount, cannot make incremental progress, and its stall "can nevertheless be indefinite." It is scoped to `EPOCH_WITHDRAW` and `REBALANCE_WITHDRAW` recovery, where a lending market cannot supply the requested liquidity. This is `EPOCH_DEPOSIT` recovery, and the distinction decides the outcome: KI-014's stall waits on market liquidity, which returns. Aave v4 maintains a non-decreasing added-asset to added-share exchange rate, so the conversion that rejects one base unit today rejects it strictly more firmly tomorrow. Waiting is a valid response to KI-014 and is not a response here.

Impact

An unprivileged user can indefinitely deny all holders access to the strategy principal with roughly two USDC of intents. The affected cohort cannot claim or cancel, later epochs cannot close, and the strategy cannot rebalance. Normal operation can resume only after a vault upgrade/state repair or an unsafe accounting override; waiting and exact recovery do not help.

The impact is a permanent lock of the entire strategy position combined with protocol-wide liveness denial, and the cost of causing it is approximately two USDC. No privileged role is involved, no external market condition is required, and nothing about the attacker's position is unusual beyond holding shares to withdraw. The one constraint on likelihood is that the epoch's aggregate net flow must land on exactly one base unit, which requires submitting intents against a known epoch aggregate rather than blindly — intents are public and the submission window is open until close, so this bounds the attack without preventing it.

Recommendations

The correction must happen before Parent commits an asynchronous remote deposit; changing the adapter's 100-unit tolerance is insufficient.

1. Add an adapter preview/minimum-deposit interface. For Aave v4, resolve the reserve's Hub and asset ID and require the asset-to-added-share preview to return at least one share.
2. Before marking a remote epoch executing, reject or reconcile a positive net flow that the target adapter cannot accept. A sub-minimum amount can remain explicitly accounted as loose dust or roll into a later epoch while the current cohort becomes claimable.
3. Classify `InvalidShares()` as terminal rather than replayable on Child. Add an authenticated Child-to-Parent abort message that accounts for the loose asset, clears recovery, and safely finalizes or reopens the epoch.
4. Add fork regressions for zero, one, and the first amount that previews to a nonzero Aave v4 share, including after the reserve exchange rate increases.

Pooling the amount with a later inbound message is not a recovery under the current implementation, because the singleton recovery guard blocks processing that later message.

Mitigation

Fixed in e7e38a6: the adapter previews the conversion and retains an unrepresentable amount as capped, tracked assets instead of letting the upstream reject it, applied to Aave v4 and v3 alike.

7.3 Medium Severity Findings

[M-01] Failed close-report retries let late depositors capture incumbent yield

Affected files

- `evm/src/vaults/ParentVault.sol#L350–L397`
- `evm/src/modules/WorkflowRouter.sol#L106–L145`
- `evm/src/libraries/vaults/ParentVaultEpochLib.sol#L180–L220`
- `evm/src/libraries/vaults/ParentVaultUserEpochLib.sol#L337–L350`

Description

`ParentVault.closeEpoch(expectedEpochNonce, tvl)` rejects reports for a different epoch, but it accepts the supplied TVL for as long as the same epoch remains open. This is unsafe for failed authenticated reports: `WorkflowRouter` forwards the report without using its report ID or enforcing freshness, while `Keystone` deliberately leaves a failed transmission retryable.

A depositor can make a correct report fail after it has been signed by cancelling the only deposit in the epoch before delivery. The failed transaction publishes the authenticated report bytes and signatures without advancing the epoch nonce. Because the epoch is then empty, later scheduled close attempts skip it; meanwhile the active strategy continues accruing yield. The attacker can later deposit and permissionlessly retry the old failed report before the next fresh close. The nonce still matches, so their deposit is priced against the pre-yield TVL.

The CRE report is truthful when signed. The attacker creates the stale state using ordinary deposit cancellation and the intended failed-report retry behavior; no incorrect CRE observation is required.

This is not the documented delayed-settlement case. `KI-007` states that when a scheduled close is missed, "Settlement allocations use TVL at the eventual close, not at the missed scheduled close time," and that the delay "does not by itself create an accounting inconsistency or direct loss of funds." Both statements fail on this path: the eventual close settles against the *missed* snapshot, and the result is a direct transfer of incumbent assets. `KI-010`'s bounded-staleness reasoning is scoped to the zero-total-shares bootstrap case and inherits the same assumption, so it does not cover this either. The individual mechanisms are documented as intentional — `closeEpoch` records that the supplied TVL is trusted and unvalidated, and `WorkflowRouter.onReport` records that it does not reject stale or replayed reports — but no accepted risk covers their composition producing shareholder loss.

Where this sits relative to the existing protections

Nothing on this path is missing or misconfigured, and every guard on `closeEpoch` behaves as designed. The call is restricted to the epoch operator role, the report must name the epoch it closes, `MIN_EPOCH_PERIOD` must have elapsed, an epoch containing neither deposits nor withdraw intents cannot be closed, and settlement rejects a TVL-to-share ratio that rounds down to zero. Every one of those checks passes during this sequence, because none of them is the thing that fails.

The epoch nonce binding is explicitly replay protection, and it does its job: a report cannot close an epoch other than the one it names. What a nonce cannot express is *which state* of that epoch it was authorized

against. The nonce advances only when a close succeeds, so deposits and cancellations change an epoch's contents freely while its identity stays fixed. A report signed against the epoch as it stood at one moment remains equally valid against entirely different contents later.

The trust placed in the reported TVL is a deliberate architectural decision rather than an oversight. The contracts do not compute cross-chain strategy value on-chain because for a remote strategy they cannot, and the consequence is documented: an incorrect TVL irreversibly corrupts epoch share accounting once a claim occurs. That boundary is drawn around *incorrect* values supplied by a trusted reporter. This finding involves no incorrect value. The reported TVL is accurate at the moment it is signed, and becomes wrong only because an unprivileged actor determines when it is applied. The missing axis is not accuracy but freshness: nothing on the path records when the value was observed, and nothing requires it still to hold.

Where the active strategy is local, the boundary is also wider than it needs to be. The Parent already observes the authoritative figure — the adapter-backed TVL it exposes through its own view function — and settlement uses the supplied value regardless.

The last component is worth stating plainly, because it is a correct protection rather than a flawed one. Refusing to close an epoch that contains neither deposits nor withdraw intents is sensible: there is nothing to settle. In this sequence it is also what keeps the stale report usable. An epoch with activity would be closed by the next scheduled report, advancing the nonce and rendering the old report permanently invalid. By emptying the epoch, the attacker relies on that guard to guarantee no fresh close can ever supersede the one they hold.

Vulnerable Scenario:

1. Existing shareholders own T assets in a local strategy and epoch n is open.
2. The attacker deposits the minimum amount, making the epoch eligible for close.
3. CRE reads nonce n and TVL T , then signs the correct close report.
4. The attacker cancels their deposit before delivery. The vault call fails because the epoch is empty, and Keystone records the transmission as failed.
5. The attacker waits while strategy yield Y accrues. With no epoch activity, scheduled close runs do not supersede the failed report.
6. The attacker deposits D and retries the published report. `closeEpoch` accepts nonce n and stale TVL T .
7. The attacker receives D / T of the old share supply rather than $D / (T + Y)$.

Impact

Existing shareholders permanently lose $Y * D / (T + D)$ of yield to the late depositor. In the PoC, a 10,000,000 USDC deposit captures 500,000 USDC of a 1,000,000 USDC incumbent yield accrual; subsequent reports, cancellation, pausing, or epoch progression cannot reprice the already-claimable shares.

The same replay settles a remote-strategy epoch against its stale snapshot, so the exposure is not confined to a locally held strategy. In that configuration the ParentVault has no locally observable value to compare against at all.

Exploitability. Realising material value requires four conditions to hold together, and three of them are outside the attacker's control. The cancellation must land between the workflow's TVL read and the report's delivery, which on a chain without a public mempool is a blind timing race — though a cheap and freely repeatable one, since cancelling too early simply means no report is signed and too late means the epoch closes normally. The epoch must then stay empty long enough for yield to accrue, because any other deposit or withdraw intent lets the next scheduled close succeed and permanently invalidates the stale report. The attacker must commit capital at least comparable to existing TVL to capture a meaningful fraction, and that capital is not flash-loanable, since exiting requires a subsequent epoch. Finally the accrued yield must be material over that window.

The second and fourth conditions work against each other: a vault quiet enough for epochs to sit empty holds too little for the accrual to be worth taking, while one holding enough for it to matter has the activity that closes the window. The figures in the proof of concept below are chosen to make the arithmetic legible rather than to represent realistic lending yields.

Recommendations

Bind each report to an observation deadline carried in its signed payload. Have the report producer append the deadline to the report body, validate it in `WorkflowRouter`, and strip it before forwarding to the vault. Because the deadline sits in the report body it falls under the existing signature digest, so no change to the Keystone forwarder is required. The check is independent of where the active strategy lives, so local and remote settlement behave identically and no validation asymmetry is introduced between them.

```
if (report.length < 36) revert
WorkflowRouter__ReportTooShort(report.length);
uint256 deadline = uint256(bytes32(report[report.length - 32:]));
if (block.timestamp > deadline) revert
WorkflowRouter__ReportExpired(deadline, block.timestamp);
bytes calldata vaultCall = report[:report.length - 32];
// existing metadata and selector checks run against vaultCall
(bool success, bytes memory returnData) = i_vault.call(vaultCall);
```

Verified against the audited commit:

Property	Result
Honest fresh report inside its deadline	Settles normally; transmission SUCCEEDED, epoch advances
The stale replay described above	Rejected; transmission stays FAILED, epoch nonce unchanged
Liveness after rejection	Preserved; a re-signed report at live TVL settles the epoch
Attacker's allocation	Priced at live TVL, capturing none of the incumbent yield
Fixture cost: 6 <code>WorkflowRouter</code> unit tests and 22 integration tests fail until they append the deadline, all with <code>WorkflowRouter__ReportTooShort</code> or <code>WorkflowRouter__ReportExpired</code> . Every one is a report-format update rather than a behavioural failure. As with any change to the signed payload, the contracts and the report producer must be deployed together.	

Selector coverage. Apply the deadline uniformly to every report the router dispatches rather than to `closeEpoch` alone. Per-selector opt-in fails silently: a command added later is unbound by default, with nothing to signal it. A uniform rule also keeps the length arithmetic relative rather than hardcoded per selector.

Choosing the window. The deadline should be generous enough to absorb ordinary reporting latency and network congestion — otherwise honest reports expire and settlement stalls — while remaining far shorter than the interval over which strategy yield becomes material. Given a roughly daily settlement cadence with a one-hour floor, a window in the minutes range leaves ample margin on both sides, but the exact figure is an operational decision.

If the destination binding recommended for the receiver-redirect finding is also adopted, both values can be carried in a single trailing block and validated together, so the report format changes once rather than twice.

Mitigation

Fixed in 14a57d6: the signed envelope carries an observation timestamp and `onReport` rejects anything future-dated or older than thirty minutes, so a stale report can no longer settle changed epoch contents against its original TVL.

[M-02] Adapter donation recovery lets an unseeded bootstrapper steal depositor principal

Affected files

- `evm/src/libraries/vaults/ParentVaultEpochLib.sol#L209–L268`
- `evm/src/libraries/vaults/ParentVaultUserEpochLib.sol#L232–L263`
- `evm/src/vaults/ParentVault.sol#L200–L224`
- `evm/src/modules/adapters/AaveV3Adapter.sol#L109–L127`

Deployment context (outside the reviewed contract scope, referenced only to establish the starting state): `evm/script/deploy/DeployParent.s.sol`, `docs/operator/DEPLOYMENT.md`.

Description

Two properties of settlement combine to let an unprivileged account take honest depositors' principal.

`ParentVaultUserEpochLib._claimShares` calculates each deposit claim against shrinking remaining-input and remaining-output pools. Every non-final claimant is rounded down, while the claimant that exhausts the pool receives the entire remainder. `ParentVault.claimSharesFor` is permissionless, so claim order is chosen by whoever calls it rather than by the claimants — an attacker can force many minimum-sized deposits to claim first and make its own deposit the exhausting claim.

Separately, `AaveV3Adapter.getTVL()` counts `aTokens` held by the adapter as strategy backing even when the value never passed through a `ParentVault` deposit, and anyone can supply underlying to Aave on behalf of the adapter. Combining a withdrawal of existing shares with new deposits so that the next epoch settles at exactly zero net flow leaves the unsolicited position in Aave while fresh `ParentVault` deposits pay the withdrawal, returning the out-of-band capital in full.

Both are bounded by how coarse the share unit is, and the share unit is coarse exactly once: at bootstrap. The parent deployment leaves both authoritative and ERC-20 share supply at zero, and neither `DeployParent` nor the deployment runbook performs the permanent seed deposit on which KI-010's bootstrap analysis relies. The first unprivileged depositor can therefore own the complete bootstrap supply, withdraw almost all of it, and leave only a few share wei outstanding — at which point a single floored claim represents material underlying value rather than dust.

This exceeds the accepted behavior in KI-008 and KI-010. KI-008 concludes that unsolicited adapter value cannot be fully recovered permissionlessly; the zero-net settlement pays it back in full.

KI-010 relies on a permanent operational seed and states that other users are not put at risk, but the audited deployment path neither creates nor requires that seed and this sequence transfers honest principal.

Where this sits relative to the existing protections

The guards around settlement work, and one of them shapes the attack rather than stopping it. Settlement rejects an aggregate allocation under which a minimum-sized deposit would mint zero shares, which caps how coarse the share unit can be made: with a one-USDC minimum deposit, one

share

wei can never be made to carry more than roughly one USDC of value. The attacker tunes the supply to sit just inside that limit rather than defeating it, so the per-claim rounding loss is bounded by that same figure. That guard's edge was clearly anticipated: an operator role exists specifically to force-cancel a deposit that would otherwise leave an epoch unclosable. The sequence below never trips it — the supply is tuned to sit inside the admissible range, so settlement proceeds normally and no operator intervention is prompted.

The shrinking-pool claim calculation is likewise deliberate and, in ordinary conditions, correct. Assigning the exact remaining balance to the claimant that exhausts a pool is what guarantees the deposit-side counters reach zero together and that no share is stranded. It is only harmful once the share unit is coarse enough that a single floored claim represents real value, and once claim order is chosen by someone other than the claimants.

The seed deposit that bounds this class is described as the operational mitigation for the accepted bootstrap-allocation issue. It is stated only in that acceptance rationale. The deployment runbook does not instruct it and the parent deployment script does not perform it, so an operator following the documented deployment procedure produces a vault with zero authoritative shares and zero token supply. Nothing in the contracts requires the seed to exist.

The consequence exceeds what the accepted issues describe. The bootstrap-allocation entry states that no other user's balance is diluted or put at risk and that the effect is a one-time transfer of dust value; here honest depositors lose a third of their principal. The third-party-supply entry concludes that unsolicited adapter value cannot be recovered permissionlessly; an exactly zero-net epoch returns it in full, because fresh deposits fund the old withdrawal while the donated balance stays in the strategy as backing.

Vulnerable scenario:

1. The attacker is the first depositor after deployment, deposits 1 USDC, and receives $1e18$ bootstrap share wei.
2. The attacker withdraws all but 300 share wei. Rounding leaves one USDC base unit in Aave.
3. The attacker supplies 199.999999 USDC directly to Aave on behalf of the adapter, making its TVL exactly 200 USDC.
4. In one epoch, the attacker deposits 100 USDC, withdraws its 300 old shares, and 100 users each deposit 1 USDC. Deposits and the old withdrawal are both valued at 200 USDC, so the close has zero net flow.
5. The attacker claims the 200-USDC old withdrawal from the new ParentVault deposits. The Aave position remains as backing, so all 199.999999 USDC supplied out of band has been recovered.
6. The attacker calls `claimSharesFor` for every victim before claiming its own deposit. Victims receive 100 share wei in aggregate and the attacker receives 200; with the large claim first, both cohorts would receive 150.
7. After an ordinary full exit, the attacker holds 334.333332 USDC from 300.999999 USDC of starting capital, while victims hold 66.666667 USDC from 100 USDC deposited.

No administrator deviation, false CRE report, CCIP failure, or Aave exploit is required. The attacker uses only public vault calls and Aave's ordinary supply-on-behalf functionality. Honest epoch closes use the live adapter TVL.

Impact

An unprivileged bootstrapper can recover all capital supplied outside ParentVault accounting and then redirect honest depositors' principal through permissionless claim ordering. In the deterministic reproduction, the attacker earns 33.333333 USDC net and the 100 depositors lose the same amount. The final vault, adapter, and share balances are all zero, so the result is realized cash rather than an unrealized valuation artifact.

Exploitability. The sequence is available in a narrow window and needs several conditions to coincide. The attacker must be the first depositor after deployment — a single, publicly observable opportunity rather than a repeatable one — and the deployment must have proceeded without a seed deposit. Reducing authoritative supply into the coarse range requires holding essentially all of it, which follows from being first but is not otherwise obtainable once other depositors are present.

The capital supplied outside vault accounting is recovered through the zero-net-flow epoch rather than forfeited, so the attacker's cost is gas and opportunity cost. The extraction is bounded in the other direction as well: settlement rejects any allocation under which a minimum-sized deposit would mint zero shares, which caps one share wei at roughly one minimum deposit of value. The transfer therefore scales with the number of separate claims rather than with the amount deposited, and it is zero where victim deposits divide the supply evenly and leave no remainder to sweep.

What outlasts the window is the share unit itself. New shares mint in proportion to existing supply, so the coarse unit established at bootstrap persists as the vault grows rather than diluting away, and every later depositor cohort remains exposed to the same bounded per-claim rounding loss for as long as the vault operates.

Recommendations

Reserve minimum-liquidity shares at first settlement. At the bootstrap allocation, carve a fixed share amount out of the newly allocated shares and mint it immediately to an address nobody controls, leaving the remainder as the claimable pool. The lock is never withdrawable, so authoritative supply can never return to the coarse range the attack depends on, and because new shares mint in proportion to existing supply the share unit stays fine-grained permanently.

This approach adds no new failure mode to settlement. The only guard it needs — rejecting a bootstrap whose allocation is smaller than the reserved amount — is unreachable while a minimum deposit is enforced, since the smallest admissible bootstrap allocates several orders of magnitude more shares than the reserve. It did not fire anywhere in the test suite.

Two integration costs should be planned for rather than discovered:

- The share-accounting property that states authoritative shares change at close by newly allocated deposit shares minus submitted withdraw shares needs to account for the bootstrap lock, which is allocated but never claimable. Its stateful harness is the only invariant affected; solvency,

deposit- and withdraw-pool reconciliation, recovery, nonce and CCIP properties all continue to hold unchanged.

- Roughly eighteen existing tests assert the exact bootstrap share amount or withdraw exactly that amount, and need their expectations adjusted by the reserved quantity. All are fixture expectations; none is a behavioural failure.

A minimum-supply guard on settlement is not a safe alternative. Rejecting any close that would leave authoritative supply between zero and a floor blocks this attack, but it introduces a settlement revert that withdraw intents can trigger. There is no operator function to cancel a withdraw intent — the existing force-cancel covers deposits only — so a withdrawer whose intent lands the supply inside the forbidden band and who declines to cancel it voluntarily leaves the epoch permanently unclosable, freezing every other deposit and intent in that epoch and preventing the next epoch from opening. That guard also rejects at least one settlement the current suite treats as correct.

Independently, remove caller-selected remainder assignment. Calculate each deposit claim from the epoch's immutable totals rather than from shrinking pools, and route the aggregate rounding remainder to a deterministic destination rather than to whichever claim exhausts the pool. This removes the value of ordering claims regardless of how coarse the share unit is, and it also closes the lower-value ordering advantage that exists on the withdraw side.

Finally, make the seed deposit real or unnecessary. If the operational seed is the intended control, the deployment runbook and deployment script should perform it rather than it existing only as a stated assumption. The minimum-liquidity reserve above makes it unnecessary, which is the more robust outcome.

Mitigation

Mitigated at deployment in 259528f: no contract change was required, with the permanent seed lock, the mandatory bootstrap procedure in `DEPLOYMENT.md` and KI-024 together addressing it, leaving the residual that nothing on-chain enforces the seed.

[M-03] Outbound CCIP recovery is reported as backing while committed to a settled epoch

Affected files

- evm/src/vaults/ChildVault.sol#L175–L177
- evm/src/vaults/ChildVault.sol#L245–L252
- evm/src/vaults/ChildVault.sol#L459–L469
- evm/src/vaults/ChildVault.sol#L569–L577
- evm/src/vaults/ChildVault.sol#L770–L780
- evm/src/vaults/ParentVault.sol#L395–L419
- evm/src/libraries/vaults/ParentVaultEpochLib.sol#L192–L219

Description

ChildVault._getTVL() always adds s_ccipSendRecovery.amount to the active adapter's TVL. For an

EPOCH_NET_WITHDRAW send failure, that recovery amount is not backing for the live share supply: it has already been withdrawn from the adapter and assigned to the preceding epoch's fixed asset-claim pool. Nevertheless, the Parent opens the next epoch while that preceding epoch remains EXECUTING, so an honest next-epoch TVL observation can include the reserved amount.

The preceding-epoch check in ParentVaultEpochLib.closeEpoch is evaluated only when the signed report

executes. If permissionless executeRecovery() succeeds after the report observes TVL but before its first delivery, the reserved assets move to ParentVault and make the preceding epoch CLAIMABLE. The report then passes the guard while settling the next epoch with the pre-recovery TVL. This assigns the same physical assets to both epochs.

For $P = 1,000,000$ USDC, let $W = 500,000$ USDC be the first cohort's withdrawal and $R = 500,000$ USDC the remaining backing:

Checkpoint	Physical location of W	Epoch allocation	Child TVL used for the next close
Epoch 2 closed	Active Child adapter	Epoch 2 records a W claim	P
Withdraw succeeds; CCIP Loose in ChildVault as send fails	CCIP_SEND(W)	Epoch 2 still owns W	adapter R + recovery W = P
Honest epoch-3 observation	Same loose recovery balance	Epoch 2 still owns W	Signed value is P
Recovery succeeds	ParentVault's reserve	epoch-2 Epoch 2 becomes CLAIMABLE	Live Child TVL falls to R
Signed report is first delivered	ParentVault's reserve	epoch-2 Epoch 3 is allocated P as well	Report supplies old P

Vulnerable scenario:

1. Two holders each own half the shares backed by 1,000,000 USDC in a remote Child adapter.
2. The first holder withdraws all its shares in epoch 2. An honest close allocates 500,000 USDC, marks epoch 2 EXECUTING, reduces authoritative shares by half, and opens epoch 3.
3. Child withdraws 500,000 USDC, but an ordinary transient CCIP fee or funding failure causes the send to fail. Child holds that amount in CCIP_SEND recovery while the adapter retains 500,000 USDC; getTVL() reports 1,000,000 USDC.
4. The second holder queues all remaining shares in epoch 3. Honest CRE samples the exposed 1,000,000-USDC TVL and signs a close report that has not yet been delivered.
5. Once the transient condition clears, any account calls ChildVault.executeRecovery(). The 500,000 USDC reaches ParentVault, epoch 2 becomes CLAIMABLE, and live Child TVL becomes 500,000 USDC.
6. The honest signed report is delivered for the first time. Its nonce is current and the preceding epoch is now claimable, so it closes epoch 3 using 1,000,000 USDC.
7. The epoch-3 command asks Child to withdraw 1,000,000 USDC from an adapter holding 500,000 USDC. Child records EPOCH_WITHDRAW(1,000,000 USDC) after the withdrawal fails.
8. The first holder claims its complete 500,000-USDC exit. Retrying recovery continues to request the impossible exact amount, while the second holder's shares are escrowed and its 500,000 USDC remains in the adapter.

No false report, forged message, privileged deviation, strategy loss, or unsolicited supply is required. The signed TVL exactly equals the value the Child exposes at the moment it is sampled. The sequence does require the TVL to be sampled while the recovery is pending and the settlement to execute after it clears; the section below sets out what currently prevents that and why the protection is fragile.

Where this sits relative to the existing protections, and what currently prevents it

The Parent's preceding-epoch guard is correct and does its job: while the reserved amount is latched on the Child, the earlier epoch is EXECUTING and no later epoch can close. The guard is not defeated in this sequence.

The recovery-inclusive TVL formula is also deliberate, and for three of the four amounts it sums it is right. Assets held in an epoch-deposit, rebalance-deposit or rebalance send recovery genuinely do back the live share supply; excluding them would understate TVL and misprice shares in the opposite direction. The documented intent is that pending recovery amounts are attributable exactly once and that the accounting must neither omit nor double-count them. The formula was built to prevent omission, and the single case it does not distinguish is the one where the amount is already committed elsewhere.

What prevents the wrong figure from being consumed today is neither of those. It is the order in which the offchain workflow performs its reads. The documented epoch cron reads the Parent's epoch state first and the strategy TVL second, and an epoch becomes claimable only once the reserved amount has actually reached the Parent — which is the same event that clears the recovery. Under that ordering the state check can only pass after the recovery has cleared, by which point the TVL read returns the corrected figure.

That ordering is a real protection, but it is not a contract guarantee and is not recorded anywhere as a safety requirement. Two properties make it fragile. First, nothing on-chain constrains it: a workflow that samples strategy TVL before the Parent's epoch state — an ordinary thing to do when consolidating reads — makes the sequence live immediately, with no on-chain signal that anything changed. Second, the two values cannot be made atomic. For a remote strategy the epoch state lives on the Parent chain and the TVL on the Child chain, so they are necessarily two reads at two different moments; consolidating the Parent-side reads into a single call does not close the gap between them, and mirroring the contract's checks offchain does not constrain when the inputs were sampled.

Impact

The second withdrawal epoch receives a 1,000,000-USDC claim against only 500,000 USDC of remaining strategy assets. The first cohort exits in full, while the remaining holder loses access to all of its shares and underlying. Ordinary recovery cannot finish because it retries the stored exact amount; later epochs and rebalances cannot progress while that epoch remains EXECUTING. Restoring liveness requires at least the 500,000-USDC shortfall from exceptional yield, recapitalization, or a contract upgrade/state repair.

Reachability. Under the currently documented workflow read order this sequence does not occur, for the reasons given above. The finding is that the Child reports a figure its own accounting knows to be committed elsewhere, and that nothing in the contracts prevents that figure being consumed — the only barrier is an undocumented offchain ordering in a component that is expected to change. The proof of concept below demonstrates what follows if the figure is ever consumed; it constructs the settlement report directly rather than modelling the workflow's read sequence.

Recommendations

Exclude only an EPOCH_NET_WITHDRAW CCIP-send recovery from share-backing TVL when an adapter remains active. That amount is already committed to the preceding withdrawal cohort. Continue counting EPOCH_NET_DEPOSIT recovery because it is newly arrived backing, and continue counting REBALANCE send recovery when the old adapter has been cleared.

```
function _getTVL() internal view override returns (uint256 tvl) {
    BaseVaultStorage storage $_baseVault = _baseVaultStorage();
    ChildVaultStorage storage $ = _childVaultStorage();

    address activeAdapter = $_baseVault.s_activeProtocolAdapter;
    if (activeAdapter != address(0)) {
+       uint256 ccipSendBacking = $.s_ccipSendRecovery.ccipTxType ==
Types.CcipTx.EPOCH_NET_WITHDRAW
+       ? 0
+       : $.s_ccipSendRecovery.amount;
        tvl = IProtocolAdapter(activeAdapter).getTVL() +
```

```

$.s_epochDepositRecovery.amount
-      + $.baseVault.s_rebalanceDepositRecovery.amount +
$.s_ccipSendRecovery.amount;
+      + $.baseVault.s_rebalanceDepositRecovery.amount +
ccipSendBacking;
    } else {
        // The adapter is cleared before a remote rebalance send; a
        failed send remains locally recoverable
        tvl = $.s_ccipSendRecovery.amount;
    }
}

```

Add a regression test that creates a real EPOCH_NET_WITHDRAW send recovery and asserts that its amount is excluded from `getTVL()` both before and after the next epoch becomes eligible to close.

The value of this change is that it removes the dependency on offchain read ordering altogether. With the exclusion in place the Child never reports a figure that is committed elsewhere, so it no longer matters in which order a consumer samples the two chains, nor whether a future workflow revision happens to preserve the current sequence.

If the ordering is instead relied upon as the control, it should be recorded as a safety requirement rather than left as an implementation detail — specifically that the Parent's epoch state must be sampled no earlier than the strategy TVL. Note that this cannot be satisfied by consolidating reads into a single call: for a remote strategy the two values live on different chains, so they are always two reads at two moments, and mirroring the contract's guards offchain constrains the checks but not the age of their inputs.

As defense in depth, bind every close report to an observation timestamp or block commitment, so a sampled figure cannot be settled arbitrarily long after it was taken.

Mitigation

Fixed in 79b5662: `getTVL` now counts a pending CCIP-send recovery only when it belongs to a rebalance, excluding epoch-withdraw amounts already reserved for a settled epoch.

[M-04] Over-capacity CCIP transfer is stored as replayable recovery, permanently locking every holder's principal

Affected files

- `evm/src/vaults/ChildVault.sol#L162–L179` (`_ccipSend` — the unconditional catch)
- `evm/src/vaults/ChildVault.sol#L296–L347, #L518–L535` (rebalance and its recovery, both bridging the full position)
- `evm/src/libraries/vaults/BaseVaultCcipLib.sol#L240–L253` (`_validateCcipSend` — no upper bound on the amount)
- `evm/src/libraries/vaults/ParentVaultEpochLib.sol#L192–L194` (previous-epoch guard)

Description

`ChildVault._ccipSend` treats every `ccipSend` failure as transient:

```
try this.tryCcipSend(bridgeAmount, destinationChainSelector, ccipTxType,
nonce, protocolId) {}
catch {
    _storeCcipSendRecovery(_baseVault, bridgeAmount,
destinationChainSelector, ccipTxType, nonce, protocolId);
}
```

The stored recovery holds the exact `bridgeAmount`, and `executeRecovery()` replays that same amount.

That is correct for a transient failure (fee spike, momentary router unavailability) and wrong for a permanent one, because a permanent failure replays into itself forever.

The distinction is one the code already makes. The line directly above the `try` reads *"Validate outside try/catch so configuration errors revert instead of being stored as recovery"*, and `_validateCcipSend` is hoisted out for exactly that reason. Configuration errors are meant to revert; only transient failures are meant to become replayable recovery. A permanently-unsatisfiable transfer is the former, and it lands on the wrong side of that line.

CCIP token pools have exactly such a permanent failure. `RateLimiter._consume` distinguishes two cases, and only the second is a wait:

Condition	Error	Clears?
<code>capacity</code> <code>requestTokens</code>	<code>< TokenMaxCapacityExceeded</code>	Never — independent of fill level and of elapsed time
<code>tokens</code> <code>requestTokens</code>	<code>< TokenRateLimitReached</code>	Yes, on refill

The capacity comparison sits after the refill step and reads only `capacity`, never the current token balance and never elapsed time. A bucket can never hold more than `capacity`, so a single transfer

larger than the lane's configured capacity is rejected permanently, no matter how long the vault waits. The rate-limit branch, by contrast, returns a `minWaitInSeconds`.

Nothing bounds the amount against that ceiling. `BaseVaultCcipLib._validateCcipSend` checks only that

the amount is nonzero, the destination selector is valid and non-local, and a counterpart vault is registered. A cross-chain rebalance withdraws `type(uint256).max` from the source adapter (`ChildVault.sol#L301`, and again at `#L528` in recovery) and bridges the entire resulting position in one message, so the amount scales with TVL and will cross any fixed ceiling as the protocol grows.

Once the send is caught, `_requireNoRecovery` rejects every other `ChildVault` entry point, so the position cannot be reduced below the ceiling by any in-protocol path. The Child is closed to further work and the assets sit in its balance.

The stranded operation also holds the Parent. The epoch whose remote leg never completes cannot leave `EXECUTING`, and `ParentVaultEpochLib` refuses to close any later epoch while it has not become `CLAIMABLE`:

```
uint256 previousEpochNonce = epochNonce - 1;
if (previousEpochNonce != 0 && $.s_epochs[previousEpochNonce].status !=
    Types.EpochStatus.CLAIMABLE) {
    revert
    IParentVault.ParentVault__EpochNotClaimable(previousEpochNonce);
}
```

Who can trigger it

The precondition is reached by the position growing past the lane's capacity, with both operator roles performing their ordinary duties and no privileged misbehaviour anywhere.

It is not limited to organic growth. `ParentVault.withdraw` is permissionless, and `executeEpochWithdraw` bridges the epoch's net withdrawal amount, so the bridged value is determined

in aggregate by users. A party willing to lock its own principal alongside everyone else's can contribute enough withdrawal volume — alone, or on top of organic withdrawals — to carry an epoch's net amount across the ceiling. That self-inflicted cost is what bounds the likelihood, not any privilege requirement.

KI-016 accepts a failed CCIP send because the caller can "simply retry the same call once market conditions allow it." That holds for the LINK-balance and router-liveness causes it considers, and not here: no change in conditions permits an over-capacity transfer, so the retry is not a retry.

The ceiling itself is CCIP token-pool configuration rather than a protocol parameter, and `RateLimiter._consume` skips the check entirely when the bucket is disabled. The in-scope mechanism is

unaffected either way: a permanently unsatisfiable send is filed as replayable recovery.

Vulnerable Scenario:

1. The strategy position on a ChildVault grows beyond the configured outbound capacity of the CCIP lane back to the parent chain.
2. A net-withdraw epoch closes, or a rebalance is initiated. The epoch operator calls the corresponding ChildVault entry point; the vault withdraws from the adapter and attempts the bridge.
3. The token pool rejects the transfer with `TokenMaxCapacityExceeded`. `_ccipSend` catches it and stores `CCIP_SEND` recovery for the full amount.
4. Every `executeRecovery()` call replays that amount into the same ceiling and reverts. No elapsed time changes the outcome.
5. `_requireNoRecovery` blocks every other ChildVault entry point, so the position cannot be reduced.
6. The Parent epoch cannot advance out of `EXECUTING`, and every later `closeEpoch` reverts `ParentVault__EpochNotClaimable`.

Impact

Every holder loses access to their principal for as long as the position exceeds the lane's capacity. The affected epoch's participants can neither claim (their epoch never reaches `CLAIMABLE`) nor cancel (KI-017 — their epoch has left `OPEN`), and no later epoch can settle for anyone else. Recovery cannot resolve it, because recovery is the thing that is stuck. Restoring service requires Chainlink to raise the lane's capacity, or a purpose-built upgrade that can rewrite the stored recovery amount.

Recommendations

Do not file a permanently-unsatisfiable send as replayable recovery. Re-throwing turns an unrecoverable vault into the atomic-revert shape KI-016 already accepts, where the epoch or rebalance rolls back cleanly and the operator can redirect or wait for a capacity change:

```
+import {RateLimiter} from "@chainlink/contracts-  
ccip/contracts/libraries/RateLimiter.sol";  
+  
    try this.tryCcipSend(bridgeAmount, destinationChainSelector,  
ccipTxType, nonce, protocolId) {}  
-    catch {  
+    catch (bytes memory err) {  
+        if (bytes4(err) ==  
RateLimiter.TokenMaxCapacityExceeded.selector) {  
+            assembly { revert(add(err, 0x20), mload(err)) }  
+        }  
        _storeCcipSendRecovery($_baseVault, bridgeAmount,  
destinationChainSelector, ccipTxType, nonce, protocolId);  
    }
```

That contains the damage but does not make the transfer possible. To keep large positions bridgeable, also either bound the position against the destination lane's capacity when selecting a rebalance target, or split an over-capacity transfer across multiple messages. Whichever is chosen, record the

dependency as an explicit environment assumption alongside ENV-002, since the current set does not state that a bridged amount must fit within the lane's configured capacity.

Mitigation

Fixed in 65d4776: a `TokenMaxCapacityExceeded` failure now bubbles instead of being stored as replayable recovery, turning the terminal state into the atomic revert already accepted elsewhere.

[M-05] Base-unit withdrawals exhaust Child LINK and pin global settlement

Affected files

- evm/src/vaults/ParentVault.sol#L37–L53, #L160–L170
- evm/src/libraries/vaults/ParentVaultUserEpochLib.sol#L187–L205
- evm/src/libraries/vaults/ParentVaultEpochLib.sol#L192–L200, #L216–L284
- evm/src/vaults/ChildVault.sol#L148–L178, #L233–L253, #L553–L604
- evm/src/libraries/vaults/BaseVaultCcipLib.sol#L178–L204
- evm/src/vaults/BaseVault.sol#L359–L364

Description

At frozen revision `a08ecdadb5aa603366ac9a342791cc532052ee52`, a user deposit is floored at one underlying token, but a withdrawal is floored only at one nonzero share unit. An ordinary holder can therefore split a single one-USDC share position into roughly one million remote withdrawal epochs whose net withdrawal is one USDC base unit each.

Every such epoch makes the Child withdraw that one atom from the active lending adapter and construct a CCIP message. `BaseVaultCcipLib._sendPacked` quotes the message, then pays the entire fee from the Child vault's shared LINK balance:

```
uint256 fee =
IRouterClient(params.ccipRouter).getFee(params.destinationChainSelector,
message);
IERC20(params.link).forceApprove(params.ccipRouter, fee);
IERC20(params.asset).forceApprove(params.ccipRouter,
params.bridgeAmount);
IRouterClient(params.ccipRouter).ccipSend(params.destinationChainSelector,
message);
```

The fee is not charged to the requesting holder, is not related to the amount bridged, and is not subject to a fee-linked minimum flow. The holder receives one atom per successful epoch, retains almost the entire share position, and can repeat once per minimum one-hour epoch without another deposit.

When the finite Child LINK reserve falls below one quote, `executeEpochWithdraw` has already successfully removed the atom from the lending adapter. Child's `_ccipSend` catches the insufficient-LINK failure and stores `CCIP_SEND` recovery rather than reverting the adapter withdrawal:

```
try this.tryCcipSend(...) {}
catch {
    _storeCcipSendRecovery(...);
}
```

The corresponding Parent epoch remains `EXECUTING`. `ParentVaultEpochLib._closeEpoch` requires the preceding epoch to be `CLAIMABLE`, so every later epoch close reverts. New intents can be

cancelled while their current epoch remains open, but no later deposit or withdrawal settlement can advance until someone externally supplies LINK and retries recovery.

Vulnerable scenario

1. The deployment has normal live share supply, incumbent principal in an active remote Child strategy, and a finite Child LINK reserve B.
2. The attacker deposits the minimum one USDC once, the ordinary remote-deposit epoch completes, and the attacker claims approximately $1e18$ shares.
3. In each subsequent epoch, the attacker withdraws only the shares whose truthful pro-rata value floors to one USDC atom.
4. Ordinary CRE closes the epoch. Parent records a one-atom remote net withdrawal and enters EXECUTING.
5. Child successfully withdraws the atom from its adapter and pays the full CCIP quote q from shared LINK. The attacker may claim the atom and repeats with the remaining shares.
6. When Child has $< q$, the next adapter withdrawal still succeeds, but `ccipSend` fails. Child stores the atom and fixed message parameters in `CCIP_SEND` recovery.
7. Parent cannot complete that epoch, and its preceding-epoch guard rejects all future closes until Child receives new LINK and `executeRecovery()` succeeds.

No privileged role, false or stale report, bootstrap/coarse supply, donation, abnormal token behavior, upstream pause, governance change, market insolvency, or impossible external behavior is used.

KI-016 considers this class on the Parent side and accepts it, recording that an underfunded LINK balance is an operational concern and that *"no attacker-controlled trigger for a CCIP send failure has been identified"*. That is the statement this sequence contradicts: the trigger is permissionless, costs one USDC once, and commits Child recovery after value has already left the lending adapter, rather than resolving through the atomic Parent-side revert that entry accepts.

The sequence relies on the documented workflow closing an epoch that has activity and has been open past `MIN_EPOCH_PERIOD`; a one-atom withdrawal is activity. A dust filter in the automation would defeat this particular path, but settlement liveness would then depend on an off-chain policy that no on-chain rule requires and nothing enforces. What the contract is missing is a fee-linked economic floor on the flow it will pay to bridge — the withdrawal minimum is one nonzero share unit while the cost of servicing it is a full cross-chain quote.

Impact

This is a permissionless, capital-efficient, persistent denial of withdrawal and deposit settlement.

- **Attacker capital:** one USDC once. Only the shares worth one atom are escrowed per recurring epoch; almost the entire position remains transferable.
- **Permanent attacker loss:** zero principal and zero LINK; normal transaction gas only. Each successful iteration redeems rather than burns economically unclaimed value.
- **Protocol loss:** one full CCIP LINK fee per one-atom withdrawal.

- **Victim impact:** after exhaustion, later settlement for the full live strategy position is blocked until external recapitalization. The PoC demonstrates this against 1,000,000 USDC of normal incumbent backing.
- **Repeatability:** the PoC executes eight consecutive one-atom withdrawals and proves that each consumes exactly one full Child fee. A one-USDC position contains roughly one million atoms.
- **Recurring gas:** the suite's gas report measured `ParentVault.withdraw` at most 103,285 gas. One unlimited share approval can serve every iteration; claiming each atom is unnecessary.
- **Cadence:** one withdrawal per minimum one-hour epoch plus ordinary CCIP/report latency.

Pinned production Router quotes at the frozen audit date were:

```
Base -> Ethereum initial 1-USDC deposit:      0.171990247115359784 LINK
Ethereum -> Base recurring 1-atom withdraw: 0.045632987993576443 LINK
```

At the recurring quote, a 10-LINK Child reserve reaches the failing request after approximately 220 withdrawal epochs (9.2 days); 100 LINK takes approximately 2,192 epochs (91.3 days). The attacker can keep draining operator top-ups using the same share position.

The identical no-attack control uses the same initially sufficient Child reserve and successfully settles an incumbent withdrawal exceeding 99,000 USDC with no recovery.

Recommendations

Charge the actual quoted CCIP fee to the epoch cohort causing the remote message. One robust design is to escrow LINK, or a conservatively oracle-priced underlying service fee, with the intent; consume the actual `getFee` amount at settlement and refund excess pro rata.

If fees remain protocol-subsidized, segregate and carry forward undersized remote flows until a fee-linked economic threshold is reached without letting those requests block unrelated users. A fixed minimum should be calibrated against the live fee rather than merely copying the one-USDC deposit minimum.

As defense in depth, quote/check the Child fee balance before withdrawing from the adapter and keep a protected reserve. Monitoring and reserve floors alone are insufficient: they detect or move the exhaustion boundary but do not remove the permissionless resource amplification.

Mitigation

Mitigated across 959087c, 93c3582 and 4864ee0: the settlement charge was removed and `closeEpoch` now reverts when a remote net withdrawal falls below one whole asset unit, with the off-chain workflow deferring settlement via the `totalShares` getter added in e3c5f25. The residual is that `closeEpoch` is operator-gated, so the on-chain revert constrains only the operator while blocking every participant in an epoch whose net remote withdrawal lands in that window, until organic flow moves it.

[M-06] CCIP token-pool fee updates silently undercollateralize remote-deposit shares

Affected files

- `evm/src/libraries/vaults/BaseVaultCcipLib.sol#L178–L204`
- `evm/src/libraries/vaults/ParentVaultEpochLib.sol#L203–L241, #L305–L318`
- `evm/src/vaults/ParentVault.sol#L383–L435`
- `evm/src/vaults/ChildVault.sol#L82–L110`

Description

Yieldcoin assumes that a successful CCIP token transfer delivers the full amount passed to `ccipSend`. That assumption does not hold for the vendored CCIP V2 token-pool semantics.

`IPoolV2.TokenTransferFeeConfig` documents that `finalityTransferFeeBps` and `fastFinalityTransferFeeBps` are deducted from the transferred asset. The token-pool owner can enable either fee for a destination chain through `TokenPool.applyTokenTransferFeeConfigUpdates`. `TokenPool.lockOrBurn` then calculates `destTokenAmount = sourceAmount - feeAmount`, and `OnRamp._lockOrBurnSingleToken` places that smaller amount in the successful CCIP message.

Yieldcoin does not bind its remote epoch accounting to the destination amount:

1. `ParentVaultEpochLib.closeEpoch` calculates new shares from the full deposit amount and immediately incorporates them into `s_totalShares`.
2. `BaseVaultCcipLib._sendPacked` sends the nominal net-deposit amount and only checks whether `ccipSend` succeeds.
3. `ChildVault._ccipReceive` deposits the smaller, authenticated post-fee amount and emits its normal success event.
4. `ParentVault.completeEpochDeposit` makes the epoch claimable without receiving or checking the deposited amount.

Every external call succeeds and the message reaches the intended chain and vault, but users receive nominal shares against reduced backing. No off-chain component participates: the divergence is created by the token pool inside an otherwise successful `ccipSend`, and consumed by the contract that mints against the amount it asked to send rather than the amount that arrived.

The same divergence is already handled on the opposite side of the protocol, deliberately. **KI-005** records that at settlement the vault overwrites its price-locked withdraw estimate with the actual amount produced by the adapter or delivered by CCIP, and uses that actual figure for distribution — the accepted rationale being that anything else strands underlying the vault cannot see. The withdraw path therefore already assumes bridged output can differ from expectation and reconciles to reality. The deposit path makes the opposite choice without stating it: shares are minted from the nominal amount at close and never reconciled against what the destination received.

The declared assumptions are asymmetric in the same direction. **ENV-007** states that the rebalance workflow calls `completeRebalance` only once the complete amount has reached and been deposited into

the target strategy, and that ParentVault does not prove that. A nonzero pool transfer fee falsifies that assumption directly for the rebalance path, because the complete amount can no longer arrive. For the epoch-deposit path there is no equivalent declaration at all, so the same divergence is neither prevented by the contract nor recorded as an accepted environmental condition.

Vulnerable Scenario:

1. The vault has 1,000,000 USDC deployed in its remote strategy.
2. The CCIP source-pool owner enables a 1% transfer fee for the destination chain using the pool's advertised owner-only configuration function.
3. A user deposits 100,000 USDC and the epoch operator closes the epoch using the truthful 1,000,000 USDC TVL.
4. Yieldcoin allocates shares for the full deposit and submits a successful CCIP transfer for 100,000 USDC.
5. The source pool deducts 1,000 USDC and places 99,000 USDC in the successful destination message.
6. The Child deposits 99,000 USDC, emits `EpochDepositToStrategySuccess`, and enters no recovery state.
7. Following the documented completion condition of a successful Child event with no recovery, the operator calls `completeEpochDeposit`.
8. The user claims the full nominal share allocation, while total strategy backing is only 1,099,000 USDC.

No Yieldcoin role acts maliciously. An ordinary external pool-governance action changes the economic meaning of an otherwise successful call.

Impact

Existing and newly minted shareholders lose underlying value because the vault issues shares against assets retained as a fee by the external source pool. The live configuration permits fees approaching the complete bridged amount, and the deficit can accumulate across every later remote net-deposit epoch without a revert or recovery alert.

The failure is silent. There is no revert, no stored recovery, no divergence between the emitted event and the state it reports, and nothing for an operator to alert on: the epoch settles normally and every subsequent one does too, with the shortfall compounding into the share price rather than surfacing as an incident. Cross-chain failures in this protocol otherwise announce themselves by halting settlement, so an operational posture calibrated on that behaviour will not detect this one.

Impact is High because this is direct loss of backing. Likelihood is Low because an ordinary user cannot configure the external pool, but its owner can enable the fee through an advertised operation. The resulting severity is Medium.

Recommendations

Fail closed before every CCIP token transfer unless the vault can prove that the current source pool will deliver the full requested amount:

1. Immediately before `ccipSend`, resolve the current `OnRamp` and source pool from the live `Router` and token registry rather than relying on constructor-time compatibility.
2. If the pool supports `IPoolV2`, read its destination-specific `TokenTransferFeeConfig` and revert if the applicable `finalityTransferFeeBps` or `fastFinalityTransferFeeBps` is nonzero.
3. Revert if the current pool or fee semantics cannot be inspected reliably.
4. Perform this validation outside `ChildVault`'s `send try/catch`, so an unsupported fee unwinds the operation atomically instead of becoming fixed-parameter recovery.

For example, `_sendPacked` should be guarded as follows, with `_requireFullDestinationAmount` resolving and validating the live dependency path described above:

```
function _sendPacked(BaseVaultStore.BaseVaultStorage storage $,
SendParams memory params) private {
+   _requireFullDestinationAmount(
+       params.ccipRouter,
+       params.destinationChainSelector,
+       params.asset
+   );
    address vault =
        _validateCcipSend($, params.bridgeAmount,
params.destinationChainSelector, params.thisChainSelector);
    // ... construct message and call ccipSend
}
```

If fee-charging pools must be supported, add an authenticated destination acknowledgement carrying the amount actually deposited. Do not finalize a remote-deposit epoch or permanently allocate its nominal shares until the Parent reconciles that amount. Any shortfall must reduce the affected epoch's share allocation rather than being socialized across the vault.

Mitigation

Fixed in 2136cc1: `completeEpochDeposit` now takes the delivered amount, rejects a value above the expected net, and scales the epoch allocation and total shares down by any shortfall, binding shares to what arrived rather than what was sent.

7.4 Low Severity Findings

[L-01] ParentVault beneficiary erases withdrawals and strands all redeemed assets

Affected files

- `evm/src/vaults/ParentVault.sol#L148–L157` (`depositFor`)
- `evm/src/vaults/ParentVault.sol#L183–L193` (`withdrawFor`)
- `evm/src/libraries/vaults/ParentVaultUserEpochLib.sol#L307–L327` (`_claimAsset`)

Description

`depositFor` and `withdrawFor` reject the zero address as a beneficiary but accept the ParentVault's own address.

With `withdrawFor(address(parentVault), shares)` the payer's shares are escrowed under the vault's

own withdraw entry, which the payer cannot cancel: `cancelWithdraw` resolves the caller's entry, the vault cannot originate a call, and there is no force-cancel-withdraw. After the epoch settles, `claimAssetFor(address(parentVault), epochNonce)` consumes the entry, burns the escrowed shares and

calls `safeTransfer(address(this), amount)`. The self-transfer leaves the balance unchanged while

the claim is fully consumed, so the payer receives nothing and no account is entitled to the assets.

Those assets are then unreachable. `_getTVL()` counts only the active adapter position and rebalance-deposit recovery, and every underlying-asset outflow — withdraw claim, cancellation refund, adapter deposit — moves an exact operation amount. With no sweep or rescue path the balance stays invisible to share pricing indefinitely, which is the idle-vault-balance outcome KI-005's rationale accepts other trade-offs to prevent.

`depositFor(address(parentVault), amount)` is the symmetric case: shares are minted to the vault, cannot be submitted for withdrawal, and strand along with their backing.

Recommendations

Reject the vault's own address wherever a `For` position is created:

```
diff --git a/evm/src/interfaces/vaults/IParentVault.sol
b/evm/src/interfaces/vaults/IParentVault.sol
@@
+   error ParentVault__InvalidBeneficiary(address beneficiary);
diff --git a/evm/src/vaults/ParentVault.sol
b/evm/src/vaults/ParentVault.sol
@@ function depositFor
    _revertIfZeroAddress(beneficiary);
```

```
+         if (beneficiary == address(this)) revert
ParentVault__InvalidBeneficiary(beneficiary);
@@ function withdrawFor
    _revertIfZeroAddress(beneficiary);
+         if (beneficiary == address(this)) revert
ParentVault__InvalidBeneficiary(beneficiary);
```

With both guards applied the parent-vault unit suite passes 277/277 and the integration suite 121/121.

Mitigation

Fixed in 750a58b: `depositFor` and `withdrawFor` both reject the vault as beneficiary.

[L-02] Inactive Aave target supply can self-seal a full rebalance recovery

Affected files

- `evm/src/vaults/ChildVault.sol:296–337` (executeRebalance, _rebalanceToNewStrategy)
- `evm/src/vaults/ChildVault.sol:364–379` (executeRecovery)
- `evm/src/vaults/BaseVault.sol:359–399` (recovery store and retry)
- `evm/src/modules/adapters/AaveV3Adapter.sol:49–98`

Description

Aave V3 lets any caller supply an asset for any `onBehalfOf` address, and separately rejects a supply that would push reserve supply above the reserve's cap. A caller can therefore mint aTokens directly to a registered Yieldcoin adapter without touching the adapter or the vault, consuming capacity that a pending rebalance needs.

With P the position in the source adapter, C the target reserve's available headroom and U an unsolicited supply credited to the still-inactive target adapter, the honest rebalance fits when $P \leq C$. An attacker chooses $U > C - P$.

`executeRebalance` then withdraws the full source position, activates the target, and attempts to deposit P . The deposit fails on the cap. The source withdrawal and the adapter change are already committed and are not rolled back, so the Child stores `REBALANCE_DEPOSIT(P)` and returns successfully:

```
source adapter      = 0
target adapter      = U
Child loose balance = P
recovery obligation = REBALANCE_DEPOSIT(P)
Parent rebalance    = REBALANCING (source -> target)
```

Recovery retries exactly P , which still exceeds the remaining headroom, so every retry fails. The distinguishing property is that the blocking U belongs to the Yieldcoin target adapter itself: no unrelated Aave supplier can exit to release it, and no Yieldcoin path can withdraw it either. `AaveV3Adapter.withdraw` is vault-only, and both vault paths that withdraw — the epoch withdrawal and a further rebalance — are excluded while recovery is pending. Recovery itself only deposits.

The retry is therefore self-sealing: its own precondition is destroyed by a balance the recovering vault holds and cannot release. Parent cannot close an epoch or start a different rebalance while its first rebalance remains in progress, so the position and all outstanding shares are immobilised. Repair requires an external Aave-governance cap increase, or a purpose-built upgrade that can unwind the target, restore the principal, clear Child recovery and reconcile Parent's pending rebalance.

An unprivileged account can immobilise the vault's entire strategy principal and every outstanding share indefinitely, repairable only from outside the protocol, using capital only slightly greater

than the target reserve's headroom above the position. That overshoot is forfeited rather than recoverable — against the smallest headroom currently targeted for Aave v3 USDC, roughly 16.5m USDC against a low initial position, an attacker would irrecoverably commit several orders of magnitude more than the value they immobilise, which is what confines the trigger to targets whose headroom sits close to the position. No trusted actor misbehaves: rebalance selection and the TVL observation are truthful throughout, and the only adversarial entry point is Aave's public supply.

Vulnerable scenario

1. The position P is deployed in Child protocol A. Protocol B's Aave V3 adapter is registered but inactive, and $P \leq C$.
2. A truthful observation reports the active source TVL. `ChildVault.getTVL()` reads only the active adapter, and rebalance selection checks APY, not target reserve capacity.
3. An unprivileged account calls `Aave supply(asset, U, targetAdapter, 0)` before the selected rebalance executes, with $P + U > C$.
4. Parent commits REBALANCING from A to B.
5. Child clears A, activates B, fails to deposit P , and stores `REBALANCE_DEPOSIT(P)`.
6. Every retry reproduces the same failure, and no Child or Parent transition can reroute.

Recovery attempts that do not resolve it

Pausing every component, force-cancelling the open deposit, unpausing in dependency order, cancelling the open withdrawal intent, and retrying gives `BaseVault__DepositFailed`. The authorised Child reroute gives `BaseVault__RecoveryAlreadyPending`. Parent close and Parent reroute both give

`ParentVault__RebalanceInProgress`. Custody and obligations are unchanged throughout. Calling `completeRebalance` is not a safe escape either: it would make Parent authoritative for a target that never received the principal.

Recommendations

Add an authenticated, state-bound abort for a failed asynchronous rebalance deposit. The Child half is straightforward; the Parent half is a new lifecycle transition. Both are needed.

Child side — roll back rather than latch an unsatisfiable retry. On a failed target deposit, withdraw whatever the target holds, restore the previous adapter as active, and redeposit the combined balance there:

```
bool success = _executeDeposit(tvlToRebalance, false, newAdapter);
if (success) {
    emit RebalanceDepositSuccess(rebalanceNonce, tvlToRebalance);
} else {
    (, uint256 reclaimed) = _executeWithdraw(type(uint256).max, false,
newAdapter);
    _baseVaultStorage().s_activeProtocolAdapter = oldAdapter;
```

```
_executeDeposit(tvlToRebalance + reclaimed, true, oldAdapter);  
emit RebalanceDepositFailure(rebalanceNonce, tvlToRebalance);  
}
```

Applied to the audited commit this prevents the terminal state above: the principal returns to a yielding position, no recovery is latched, and the unsolicited supply is absorbed into vault accounting instead of sealing the retry. It is a control rather than a shipping patch — it writes the active adapter directly instead of re-resolving it, does not distinguish unsolicited surplus from vault-originated principal, and is not bound to the stored rebalance nonce, source, target and amounts. A production version should do all three.

Parent side. Even with the rollback, Parent remains REBALANCING with no way to stand down, which continues to block epoch settlement. Closing that requires an authenticated failure acknowledgement letting Parent clear the pending rebalance without marking the failed target active. This has not been prototyped here, because it is a new transition across both vaults whose authentication needs designing rather than a local change.

Reverting the failed deposit instead of storing recovery is not sufficient on its own. It preserves the source position, but the blocking supply remains and Parent is still REBALANCING, so it trades an immobilised principal for an immobilised lifecycle without providing an abort.

Selection-time capacity check. Accounting for reserve caps, utilisation and available headroom before initiating a rebalance keeps the required overshoot large enough that the attack stays uneconomical, and is a change to target selection rather than to vault logic. Because the rating above depends on that margin holding, this is the mitigation the severity is conditioned on.

Interim measure. Until both halves exist, do not schedule asynchronous rebalances into adapters whose underlying protocol lets third parties create positions for the adapter address. An off-chain capacity preflight is not sufficient on its own, because the observation can go stale between selection and Child execution.

When to re-rate. The cost ratio moves against the protocol as it grows, because the attacker's cost tracks the reserve's remaining headroom while the damage tracks the position. Below $\text{position} = \text{headroom} / 2$ the attacker pays more than the value frozen; above it they pay less, and against the tightest currently targeted headroom that crossover sits near 8m USDC of position on that chain. Evaluate against live headroom at selection time rather than against the cap, since ordinary third-party supply consumes headroom with no attacker involvement, and treat any candidate target whose headroom is close to the position as in scope irrespective of absolute size.

Mitigation

Acknowledged and documented as KI-027 in 7b97b61 and 3a8d0c0, which records the margin between the position and live headroom as the condition the rating rests on; nothing on-chain enforces that margin.

[L-03] Report-driven completion of a Parent-local rebalance target can lock backing in transit

Affected files

- `evm/src/vaults/ParentVault.sol#L513–L522` (`completeRebalance`)
- `evm/src/vaults/ParentVault.sol#L311–L344` (`_ccipReceive`, inbound rebalance delivery)
- `evm/src/libraries/vaults/ParentVaultRebalanceLib.sol#L180–L207` (`_finalizeRebalance`)
- `docs/operator/OPERATIONS.md` (manual rebalance completion procedure)

Description

A rebalance whose target is a Parent-local strategy completes itself. When the inbound CCIP message arrives, `_ccipReceive` sets and funds the local adapter and then calls `_finalizeRebalance` directly. No external completion call is needed on that route, and none should be made.

`completeRebalance` does not know this. It forwards `s_rebalance.pendingStrategy` to `_finalizeRebalance` with no check that the target is remote, and its own guards cannot detect the problem: the nonce matches, the state is `REBALANCING`, the vault is unpaused, the caller holds `REBALANCE_OPERATOR_ROLE`, and no recovery is active. All of these hold while the principal is still in CCIP custody.

Calling it during that window records the local target as the active strategy without ever funding its adapter. That overwrites `s_rebalance.activeStrategy.chainSelector`, which is the exact value

`_ccipReceive` uses to authorise the inbound message. When the authentic delivery arrives it is rejected with `BaseVault__InvalidSourceChainSelector(child, parent)`. The check runs before any

recovery handling, so the transaction reverts atomically and stores nothing.

The route is also the one route where the operator has nothing to verify. On a Parent-to-Child or Child-to-Child rebalance the destination child emits `RebalanceDepositSuccess`, so completion can be gated on observing it. A Child-to-Parent rebalance emits no such event — the deposit happens on Parent itself — so the confirmation step that protects every other route is undefined here, and nothing on-chain distinguishes the two cases for the caller.

ENV-007 already records that `ParentVault` does not prove the offchain facts behind a `completeRebalance` call. This finding is narrower: on this route the call is not merely unproven, it is never correct.

Parent reports zero vault-visible TVL while every share remains outstanding, the source child holds no position and no adapter, and the authentic transfer cannot be delivered: depositors lose access to the entire strategy principal until an upgrade or an out-of-band custody intervention restores it. No adversary participates at any point — the sequence requires a privileged operator to invoke a legitimate control during a specific window, and an operator who inspects `getTVL()` or the active adapter before completing will not proceed — so it is rated Low on likelihood, and reported rather

than dropped because the consequence is unrecoverable through the protocol's own interfaces and the guard that prevents it is three lines.

Vulnerable scenario

Manual completion is a documented control. KI-013 describes a rebalance held in REBALANCING while operators reconcile the destination result, and OPERATIONS.md gives the procedure for finalising it by hand under a temporary role grant. The sequence below uses that procedure at the wrong moment.

1. The active strategy is on a child. A rebalance back to a Parent-local strategy is initiated.
2. The source child withdraws the full position, clears its active adapter, and sends the principal to Parent over CCIP. Parent remains REBALANCING; both recovery modes are NONE.
3. While the message is in custody, an operator holding REBALANCE_OPERATOR_ROLE calls `completeRebalance` with the current nonce. Every guard passes.
4. Parent clears the rebalance, advances the nonce, and records the Parent-local protocol as active with `s_activeProtocolAdapter` still unset.
5. The authentic delivery reverts on the source-chain check and the principal stays in CCIP custody.

Recovery attempts that do not resolve it

- Retrying delivery reproduces the same rejection, because the state it depends on was overwritten.
- `executeRecovery` reverts on both vaults with `BaseVault__NoPendingRecovery`; the rejected delivery stored nothing.
- A further rebalance rolls back, because the recorded local active adapter is `address(0)` and the withdrawal step fails.
- `setInitialActiveProtocolAdapter` is one-shot and already consumed.
- No setter exists for the active adapter or for recovery state, and `setCrosschainVaults` does not help: the failing comparison is against `s_rebalance.activeStrategy.chainSelector`, not the crosschain registry.

Recommendations

Reject report-driven completion of a Parent-local target. That route is finalised by `_ccipReceive` on delivery, so refusing it here removes the only way to clear the state that authorises the inbound message:

```
diff --git a/evm/src/interfaces/vaults/IParentVault.sol
b/evm/src/interfaces/vaults/IParentVault.sol
@@
    error ParentVault__NoRebalanceInProgress();
+   error ParentVault__LocalRebalanceCompletesOnCcipReceipt();

diff --git a/evm/src/vaults/ParentVault.sol
b/evm/src/vaults/ParentVault.sol
@@
```

```

function completeRebalance(uint256 expectedRebalanceNonce) external
... {
    _requireNoRecovery(_baseVaultStorage());
    Types.Rebalance storage s_rebalance =
    _parentVaultStorage().s_rebalance;
+     if (s_rebalance.pendingStrategy.chainSelector ==
i_thisChainSelector) {
+         revert ParentVault__LocalRebalanceCompletesOnCcipReceipt();
+     }
    _finalizeRebalance(expectedRebalanceNonce,
s_rebalance.pendingStrategy);
}

```

Any automated completion handler should additionally submit the nonce carried by `RebalanceDepositSuccess` rather than reading the current nonce from Parent, so a stale event cannot be applied to a later rebalance.

When to re-rate. Two changes would remove the operator judgement that currently bounds likelihood: an automated completion handler that derives its nonce from live Parent state rather than from the nonce carried by the triggering event, and Parent-local strategies becoming a routine rebalance target rather than an occasional one.

Mitigation

Fixed in 8ecd597: `completeRebalance` now rejects a Parent-local pending strategy, which is finalised on CCIP receipt instead, and `OPERATIONS.md` no longer points an operator at that route.